1. Какво е операционна система?
1.1 Упражнение 1

2. Отговорности на ядрото

Видове софтуер

Какво е системно програмиране?

Видове софтуер

Какво е системно програмиране?

Компонент	**Интерфейс**	**Функции**
Приложна програма	Към потребителя - конзола, графично	Бизнес логика
Библиотеки за общо предназначение	Прост	Общи - низове, памет, математика и т.н.
Библиотеки за достъп до хардуер	Логическо устройство	Управление на конкретен хардуер

Какво е операционна система?

- Няма една общоприета дефиниция. По-лесно е да я опишем чрез това, което прави:
  - управлява хардуера на системата;
  - дава на програмите *непряк* достъп до него чрез интерфейс (API) - и чрез същия интерфейс ги ограничава.

Ограничения чрез интерфейса

- `open()` - ОС проверява правата за достъп до файла и може да откаже с грешка. `read()` и `write()` после работят само с вече отворения файл.
- `malloc()` и `free()` **не** са системни извиквания, а функции от `libc`. Когато им трябва още памет, те я искат от ядрото с `brk()` или `mmap()` - и ядрото решава дали да я даде.
- Ако програма достъпи памет, която не е нейна, процесорът сигнализира на ядрото и то я спира - `Segmentation fault` (сигнал `SIGSEGV`).

Какво е операционна система?

- Операционната система (ОС) е софтуер, който споделя и разпределя ресурсите на хардуера между различни програми.
- Тези програми получават възможност от ОС да не се съобразяват с останалите агенти на системата, а само със самата ОС.
- Споделянето на ресурсите се осъществява чрез *изолация* и/или чрез *заявки*.
- *Изолацията* е метод, чрез който ОС показва даден ресурс само и единствено на неговия *собственик*.

Какво е операционна система?

- *Методът на заявките* изисква изрично обръщения на всички програми към един и същ ресурс едновременно. Този вид споделяне бива два вида - *последователно* (или блокиращо, или синхронно) и *паралелно* (или асинхронно). При първото програмата блокира, докато не завърши изпратената заявка, докато при второто заявката се поставя в опашка за изпълнение и програмата няма контрол в какъв момент точно ще бъде изпълнена.

Какво е операционна система?

Ресурс	**Абстракция**	**Метод на споделяне**
Процесор	Нишки	Изолация (и времеделене)
Памет	Адресно пространство	Изолация
Диск	Файлова система	Заявки
Мрежа	Сокети	Заявки
Принтер	Логически принтер	Заявки (асинхронни)

Какво е операционна система?

Изискванията към една ОС още включват -

- Скорост - цената за изпълнение на ОС трябва да бъде минимална.
- Надеждност - от издръжливостта на ОС зависят и приложенията.
- Изолация между изпълняващите се приложения с цел сигурност. Същевременно, начин за споделяне на данни между приложения с цел кооперация.
- Абстрахиране на конкретния хардуер с цел преносимост.
- Удобен интерфейс, за да бъде лесна за употреба.

Многозадачност

- Очакваме много програми да работят едновременно, но едно процесорно ядро изпълнява инструкциите последователно - една програма в даден момент.
- Илюзията се постига с *времеделене*: таймер предизвиква прекъсване на всеки няколко милисекунди, ядрото получава управлението и може да даде процесора на друга програма.
- Програмите се редуват толкова често, че изглеждат едновременни - това е *конкурентност*, не *паралелизъм*.
- Многоядрените процесори наистина изпълняват няколко програми паралелно - и на всяко ядро отново има времеделене.

Многопотребителност

- Един компютър често се ползва от няколко потребители - понякога едновременно, например сървър, към който се свързват по SSH.
- ОС знае кой потребител е стартирал всеки процес и кой притежава всеки файл.
- Така пази данните и програмите на всеки потребител от останалите - това е изолация, но между хора, а не само между програми.

Структура на ОС

След като сме описали задачите, които една ОС трябва да решава, нека опитаме да си представим как изглежда горе-долу една такава.

Нека разгледаме един крайно опростен пример. Имаме приложения, които искат да пишат по диска, за да запазват потребителски данни. ОС, в която се изпълняват, предоставя интерфейс за работа с диска - приложенията извикват `disk_write(sector, data)` и ОС се грижи за магията.

Имаме две непосредствени следствия-проблеми. От една страна това означава, че приложение A може да реши да

Структура на ОС

използва сектор 42 и приложение Б да си хареса същия сектор. Така те ще бъдат в постоянно състояние на *състезание* (на англ. race condition) за данните на сектор 42.

От втора страна, щом това е просто функция, която може да се извика от всеки, то системата е много лесна за компрометиране - произволна програма може да изтрие данните на потребителя. Със сигурност нежелан ефект!

Достигаме до много важен извод - ОС **не** е просто библиотека с функции, които се „грижат за хардуера“. *Откъде точно произлиза разликата обаче?*

Структура на ОС

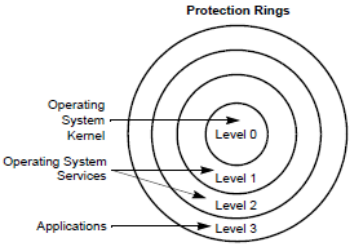
Разликата е, че не всяка програма трябва да може да пише по диска каквото и където си пожелае. Това трябва да бъде допустимо при силни ограничения - все пак не бива да го забраняваме; дискът наистина е за писане. *Как решаваме този проблем?*

Понеже разбираме, че трябва дискът да бъде споделен между множество програми, си спомняме, че това се случва чрез *заявки* към *абстракцията* на диска, а именно - *файловата система*. Така решаваме единия проблем.

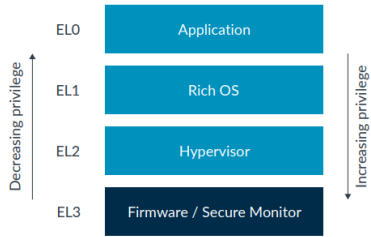
Структура на ОС

Вторият проблем, който представихме, се отнася до сигурността на ОС. Решаваме го по следния начин. Въвеждаме понятието ниво на привилегия (на англ. privilege level или още protection rings). Това е свойство на изпълняваща се програма - казваме, че се изпълнява с някакво ниво на привилегия.

Структура на ОС



Фигура: x86 Нива на привилегия



Фигура: ARM Нива за изключения

x86 има 4 нива, но Linux и Windows използват само две: ниво 0 за ядрото и ниво 3 за приложенията.

Структура на ОС

И тогава решението на проблема ни е много просто - за да изпълнява `disk_write()` (от предходния ни пример), изискваме програмата да се изпълнява с ниво на привилегия, което позволява това.

Както уточнихме вече обаче приложенията не се изпълняват с ниво на привилегия, което позволява това. И това е по дизайн.

Нуждаем се от метод за „промяна на нивото на привилегия“ - изпълняваме си приложението, вършейки позволени операции (математически операции, работа с низове, взаимодействие с потребител), и в някакъв момент искаме да извикаме

Структура на ОС

„забранената“ функция `disk_write()`. След като функцията приключи работата си, искаме нивото на привилегия да се смени отново - този път към началното си състояние.

Този механизъм се нарича системно извикване (на англ. system call или накратко syscall). Чрез него потребителско приложение инициира работа за ОС. Казваме, че приложението се изпълнява в потребителски режим (на англ. user mode), а ОС - в режим на ядрото (на англ. kernel mode). Онази част от ОС, която се изпълнява в режим на ядрото, наричаме ядро (на англ. kernel). При изпълнение в режим на ядрото програмата има

Структура на ОС

достъп до цялата системна памет и всички инструкции на процесора, докато при потребителския режим ОС се налагат ограничения.

Нека имаме едно примерно системно извикване `S()`. Технологията на извикване на `S()` е следната. Когато потребителско приложение A желае да изпълни `S()`, то трябва да задейства софтуерно прекъсване „капан“ (на англ. trap software interrupt), при което процесорът ще съхрани състоянието на A, ще премине към режим на ядрото и ще премине към изпълнение на обработка на софтуерното прекъсване. При

Структура на ОС

обработката на софтуерното прекъсване ще се изпълни работата на `S()`, след което процесорът ще изпълни специална инструкция за излизане от капана, ще възвърне състоянието на приложението и то ще продължи работата си.

Забелязваме обаче един важен детайл. При извикването на `S()` не можем просто да посочим неговия адрес. *Защо?*

Например, ако желаем да отворим файл, за който нямаме права, можем да посочим адрес, съответстващ на функцията след проверката за правоспособност. Така получаваме способност за отваряне на произволен файл, заобикаляйки

Структура на ОС

всякакви проверки за сигурност. Това се нарича уязвимост (на англ. vulnerability).

Решението на този проблем е просто. Ядрото използва фиксирана таблица за своите системни извиквания. Потребителско приложение уточнява единствено индекса на системното извикване, което иска да се изпълни, а ядрото намира адреса на съответстващата функция при обработката на софтуерното прекъсване.

Така изяснихме общата постановка на комуникация между ОС и приложенията, които ОС управлява.

Структура на ОС

Примери за системни извиквания в Linux са `write`, `mmap`, `shmat` и `dup`. На x86-64 са около 400. Може да се запознаете по-обстойно с всеки от тях, използвайки командата `man 2 <syscall>`. Можете да разгледате системните извиквания, които Linux ядрото предоставя, търсейки макрото `SYSCALL_DEFINE`*x*, където *x* е цяло число от 0 до 6 (броя параметри) в хранилището на кода - torvalds/linux.git или в индексиращата система Bootlin. Да отбележим, че когато извикаме например `write` в своя програма това **не** е системното извикване, както очакваме. Всъщност това е функция, дефинирана в стандартната

Структура на ОС

библиотека `glibc`, която извършва описания от нас процес с поставянето на число на определено място и предизвикване на софтуерно прекъсване. Такива функции се наричат обвивка (на англ. wrapper или jacket).

В macOS ситуацията е подобна на тази в Linux, поради споделения им произход. Можете да разгледате системните извиквания в хранилището на кода - apple/xnu.git. Там вместо подход с макроси, използват генератор на код.

В Windows обичайно се използват различни имена на понятията, но фундаментално идеята е същата. Наместо

Структура на ОС

системно извикване, популярният термин е системна услуга (на англ. system service). Технологията на извикване е сходна - числото, подадено от извикващия се използва от *диспечера на системни услуги*, за да индексира таблица със самите функции, имплементиращи системното извикване. Самите системни извиквания са недокументирани официално, но до известна степен следват т.нар. Native API, имплементиран в `NTDLL.dll`, което е „най-ниската точка“, изпълняваща се в потребителски режим. Примери за системни услуги в Windows са `NtReadFile` и `NtWaitForSingleObject`.

Ядрото

- Ядрото е единствената част от ОС, която работи в режим на ядрото - всичко останало работи в потребителски режим.
- То управлява хардуера пряко, включително чрез драйверите на устройствата.
- В Linux и Windows драйверите работят в режим на ядрото - в Linux често като модули, зареждани при нужда. Грешка в драйвер може да срине цялата система.
- В микроядрата (MINIX 3, QNX) драйверите са отделни програми в потребителски режим - по-бавно, но по-надеждно.

Системни извиквания - примери

Група	Системни извиквания
Процеси	`fork`, `execve`, `waitpid`, `exit`
Файлове	`open`, `close`, `read`, `write`, `lseek`
Файлова система	`stat`, `mkdir`, `rmdir`, `link`, `unlink`, `mount`, `chmod`

- В C ги извикваме чрез обвивките от `libc` със същите имена. Понякога обвивката вика друго извикване - например `waitpid()` в Linux на x86-64 използва `wait4`.
- Функциите `execl()`, `execvp()` и т.н. са обвивки около едно и също извикване - `execve`.

Обвивка (shell)

- Обвивката е програмата, чрез която потребителят работи с ОС - приема команди и ги изпълнява чрез системни извиквания към ядрото.
- Може да е текстова (`bash`, `zsh`, PowerShell) или графична (GNOME Shell, Windows Explorer).
- Освен нея ОС включва системни библиотеки (`libc`), помощни програми (`ls`, `cp`, ...) и фонови услуги.
- В GNU/Linux Linux е ядрото, а GNU дава почти всичко останало - `libc`, основните команди, `bash`, компилатора.

Упражнение

Забележка: Понеже все още не сме преминали през темите относно C, задачките ще изискват само най-елементарни познания и вярвам, че ще се справите ☺.

- Напишете програма, която приема от стандартния си вход три аргумента - име на файл, отместване във файла и низ. Използвайки обвивките на системните извиквания за работа с файл - ОТВАРЯНЕ, ПИСАНЕ и ЗАТВАРЯНЕ, програмата записва на посоченото отместване данните във файла и приключва.

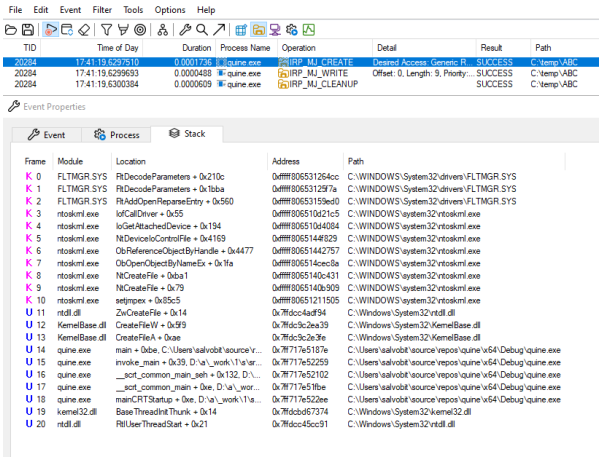
Отг. Под Linux и Mac OS използвайте open, write и close, а под Windows - CreateFileA, WriteFile и CloseHandle.

Упражнение

- Използвайте програма за проследяване на системните извиквания, за да разгледате хода на програмата си. В Windows използвайте `Procmon`, в Mac OS - `sudo dtruss`, а в Linux `strace`. Ако имате други предпочитания, нямам против.
- *Забележка (Windows):* Това, което се вижда в колоната „Operation“ в `Procmon` не са системните услуги. Същевременно при *<десен клик>* $\rightarrow$ *Stacks* се виждат ясно всички извиквания, които се пораждат от функцията, както и това в какъв момент точно се преминава към режим на ядрото.

```
1377/0x3ebc:      3829       2      1 mprotect(0x104AEC000, 0xC8, 0x1)     = 0 0
1377/0x3ebc:      3831       2      0 mprotect(0x104AFD000, 0x1000, 0x3)     = 0 0
1377/0x3ebc:      3834       2      0 mprotect(0x104AFD000, 0x1000, 0x1)     = 0 0
1377/0x3ebc:      4007       2      0 issetugid(0x0, 0x0, 0x0)     = 0 0
1377/0x3ebc:      4336       6      3 getentropy(0x7FF7BB409510, 0x20, 0x0)     = 0 0
1377/0x3ebc:      4463      15     12 getattrlist("/Users/salvobit/Desktop/a.out\0", 0x7FF7BB409CB0, 0x7FF7BB409CCC)     = 0 0
1377/0x3ebc:      4483      18     17 access("/Users/salvobit/Desktop\0", 0x4, 0x0)     = 0 0
1377/0x3ebc:      4497      14     12 open("/Users/salvobit/Desktop\0", 0x0, 0x0)     = 3 0
1377/0x3ebc:      4500       3      1 fstat64(0x3, 0x7F90A7704490, 0x0)     = 0 0
1377/0x3ebc:      4502       2      0 csrctl(0x0, 0x7FF7BB409ECC, 0x4)     = 0 0
1377/0x3ebc:      4504       3      1 fcntl(0x3, 0x32, 0x7FF7BB409BC0)     = 0 0
1377/0x3ebc:      4507       2      1 close(0x3)     = 0 0
1377/0x3ebc:      4521      10      8 open("/Users/salvobit/Desktop/Info.plist\0", 0x0, 0x0)     = -1 Err#2
1377/0x3ebc:      4539       3      1 proc_info(0x2, 0x561, 0xD)     = 64 0
1377/0x3ebc:      4555       3      1 csops_audittoken(0x561, 0x10, 0x7FF7BB409F10)     = -1 Err#22
1377/0x3ebc:      4571      10      7 sysctl([unknown, 3, 0, 0, 0, 0] (2), 0x7FF7BB40A250, 0x7FF7BB40A248, 0x7FF813739BA5, 0x15)     = 0 0
1377/0x3ebc:      4574       3      1 sysctl([CTL_KERN, 137, 0, 0, 0, 0] (2), 0x7FF7BB40A2E0, 0x7FF7BB40A2D8, 0x0, 0x0)     = 0 0
1377/0x3ebc:      4761     349    138 open("/tmp/ABC\0", 0x602, 0xDAA8)     = 3 0
1377/0x3ebc:      4781      20     18 write(0x3, "testtestov\0", 0xA)     = 10 0
1377/0x3ebc:      4845      64     63 close(0x3)     = 0 0
```

Фигура: Dtruss (откъс)



Фигура: Procmon (стек)

Отговорности на ядрото

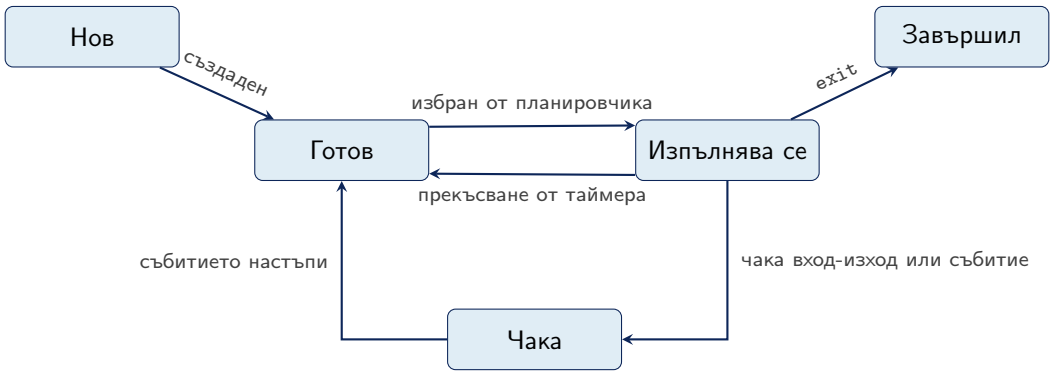
Ядрото стои между програмите и хардуера и отговаря за:

- управлението на процесите;
- управлението на паметта;
- управлението на входно-изходните устройства;
- управлението на устройствата за съхранение;
- защитата на системата.

Процеси

- Процесът е програма по време на изпълнението ѝ. От една програма може да има много процеси едновременно - например два отворени терминала.
- Всеки процес има номер - PID, положително цяло число.
- Нов процес се създава с `fork()` - копие на родителя. Така процесите образуват дърво.
- С `execve()` процесът сменя програмата, която изпълнява.
- Първият процес, `init` (PID 1, днес обикновено `systemd`), се стартира от ядрото - всички останали произлизат от него.

Състояния на процес



Смяна на състоянията

- *Готов* $\rightarrow$ *Изпълнява се* - планировчикът на ядрото му дава процесорно време.
- *Изпълнява се* $\rightarrow$ *Готов* - времето му изтече. Ядрото запазва състоянието му (смяна на контекста, на англ. context switch), за да продължи по-късно оттам, където е спрял.
- *Изпълнява се* $\rightarrow$ *Чака* - процесът сам чака нещо: вход от потребителя, данни от диска. Само изпълняващ се процес може да започне да чака.
- *Чака* $\rightarrow$ *Готов* - събитието настъпи и процесът отново чака само процесор.

Край на процес

- Процесът завършва сам (`exit`) или когато получи сигнал.
- `kill <pid>` изпраща сигнал - по подразбиране `SIGTERM`, който програмата може да прихване и да завърши спокойно. `SIGKILL` (`kill -9`) не може да бъде прихванат.
- Децата не завършват заедно с родителя си. Остават „сираци“ и получават нов родител - `init` (в Linux и процес, заявил, че осиновява сираци).
- ОС дава и механизми за комуникация (обмен на данни) и синхронизация между процесите - ще ги разгледаме по-нататък.

Управление на паметта

- RAM е огромен масив от байтове с адреси. За да бъде изпълнена, програмата трябва да е в паметта.
- Ядрото:
  - заделя и освобождава памет за процесите;
  - следи коя част от паметта на кой процес принадлежи;
  - решава кои данни да стоят в RAM и кои да се преместят на диска, когато паметта не стига.
- Всеки процес вижда собствено (виртуално) адресно пространство - процесорът превежда адресите по таблици, поддържани от ядрото.

Управление на вход-изхода

- Всяко устройство има драйвер, който знае как да работи с неговия контролер.
- Ядрото дава *единен интерфейс* за цели видове устройства: в Unix устройствата са файлове в `/dev`, с които се работи с `read()` и `write()`. Програмата не се интересува от модела на устройството.
- Само ядрото има достъп до устройствата - инструкциите за вход-изход и адресите на контролерите са недостъпни в потребителски режим.

Буфериране, кеширане, спулинг

- *Буфер* - памет, в която данните чакат по пътя между устройството и програмата. Изглажда разликите в скоростта и в размера на порциите (дискът работи с блокове).
- *Кеш* - копие на често използвани данни в по-бърза памет. Например ядрото държи прочетените части от файлове в RAM, за да не чете диска отново.
- *Спулинг* - заданията към устройство, което обслужва само едно наведнъж, чакат в опашка. Когато няколко програми печатат едновременно, принтерът получава документите цели, един след друг.

Управление на съхранението

- Устройствата за съхранение се различават по скорост, обем и начин на работа. Ядрото скрива разликите чрез две абстракции - *файл* и *файлова система*.
- Файлът е именувана поредица от байтове. Файловата система определя къде на устройството се намират те, а какво означават - решава програмата, която ги чете.
- Файловата система подрежда файловете в директории и пази правата за достъп. В Unix това са четене, писане и изпълнение - поотделно за собственика, групата и всички останали.

Защита

- **Хардуерна**:
  - режимите на процесора - привилегированите инструкции се изпълняват само в режим на ядрото;
  - защита на паметта - всеки процес достъпва само своята памет (в най-простия вариант чрез регистрите BASE и LIMIT, днес - чрез таблиците на страниците);
  - таймерът - ядрото си връща процесора, дори програма да зацикли.
- **Софтуерна**: потребители и групи, права за достъп до файловете, проверка на всяко системно извикване.

Обобщение

- Изложихме основните задачи, които решава една ОС.
- Уточнихме какво отличава ОС от библиотека за общо предназначение.
- Посочихме двата метода, чрез които ОС споделя ресурсите на машината.
- Разгледахме основната техника, използвана от ОС, за да изолират изпълнението на приложенията от хардуера.
- Видяхме как ОС постига многозадачност и многопотребителност.
- Разделихме ОС на ядро и потребителско пространство и изброихме отговорностите на ядрото.

За четене вкъщи

- Remzi H. Arpaci-Dusseau и Andrea C. Arpaci-Dusseau. *Introduction to Operating Systems*. `https://pages.cs.wisc.edu/~remzi/OSTEP/intro.pdf`. Chapter from Operating Systems: Three Easy Pieces, Version 1.10. 2018
- Remzi H. Arpaci-Dusseau и Andrea C. Arpaci-Dusseau. *Mechanism: Limited Direct Execution*. `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-mechanisms.pdf`. Chapter from Operating Systems: Three Easy Pieces, Version 1.10. 2018
- David Drysdale. *Anatomy of a system call, part 1*. `https://lwn.net/Articles/604287/`. Article from LWN.net. 2014

Край